

Qwen3-Coder-Next Tier 3

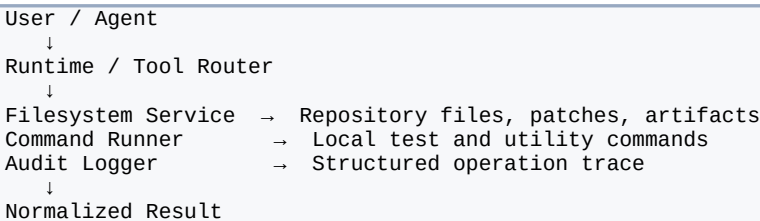
Part 2 Detailed Implementation Specification

Filesystem + Local Tooling

Engineering intent

This part defines the local execution surface that every later agent depends on: safe filesystem access, command execution, artifact capture, and normalized tool responses. It is the first place where the abstract architecture becomes a usable machine.

Field	Value
Part	2
Name	Filesystem + Local Tooling
Primary outcome	A safe local tool layer for reading, writing, diffing, and executing commands inside the workspace
Depends on	Part 1 Foundation runtime
Enables	Planner, Research, Coding, Testing, Review, and Recovery stages



Completion definition

Part 2 is complete only when an engineer can point the runtime at a local repository, read and modify files safely, execute allowed commands, capture artifacts, and verify every operation through tests and logs.

1. Purpose

This part exists to turn the runtime into a trustworthy interface to the local machine. The architecture already assumes agents can inspect code, edit files, run commands, and collect outputs. None of that is meaningful until the low-level local tooling is stable, safe, and predictable.

The problem it solves is basic but fatal when ignored: if the system cannot reliably resolve paths, enforce repository boundaries, capture command results, and preserve artifacts, later agents will act on corrupted assumptions. Planning, research, testing, and recovery all depend on a local execution layer that does not improvise.

This part is a dependency for every subsequent stage that needs real repository interaction. The planner needs file summaries. The research agent needs document access. The coding agent needs safe writes and diffs. The testing agent needs command execution. The evaluator and recovery agents need exact operation history, not guesswork.

Core principle
Part 2 is not about clever automation. It is about making the machine obey the same boring rules every time, which is apparently revolutionary in software.

2. Scope

Included in this part are the foundation services for repository-aware local execution: repository root resolution, path normalization, read operations, write operations, patch and diff generation, command execution, artifact capture, operation logging, and a stable internal tool interface.

Included	Excluded
Path resolution and workspace boundaries	Planning logic and task decomposition
File read/write/patch operations	Research retrieval and source ranking
Command execution and result capture	Memory persistence and retrieval
Artifact storage and manifests	Testing intelligence and self-healing loops
Structured operation logs	Evaluator scoring and judge logic
Tool schemas and adapters	Repository knowledge graph indexing
Local safety policies and command allowlists	Context compression and multi-agent orchestration

Excluded from this part are higher-order behaviors such as planning policies, research ranking, long-term memory, quality evaluation, recovery strategies, and repository semantic indexing. Those belong to later stages and must not be mixed into the low-level local tooling layer.

Boundary rule
If the feature decides what the agent should do, it is not Part 2. If the feature makes the local machine safely do what it is told, it probably is.

3. System Overview

The complete flow is intentionally narrow. A higher-level agent requests a local operation. The runtime validates the request, resolves the target inside the workspace boundary, executes the requested action, captures outputs, writes the audit trail, and returns a normalized response.



The same flow applies whether the request is a read, a write, a patch, a diff, or a shell command. The implementation must keep these operations distinct internally while presenting a uniform result shape to callers.

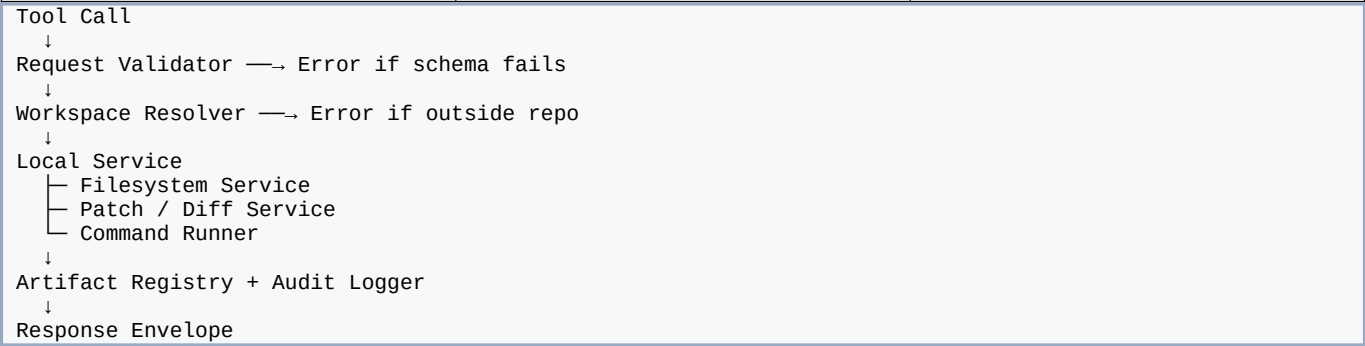
Stage	Role	What it prevents
Validation	Rejects malformed or unsupported requests	Ambiguous tool usage
Path resolution	Maps requested paths into workspace-safe paths	Traversal outside the repo

Execution	Performs the requested local action	Ad hoc behavior
Capture	Records outputs and artifacts	Lost results and undocumented changes
Audit	Writes a structured event trail	Invisible side effects

4. Internal Architecture

The internal architecture is built around a small number of deterministic services. Each service owns one job and exposes a strict interface. The runtime layer coordinates them, but no service is allowed to silently bypass policy or mutate shared state without logging.

Component	Responsibility	Primary interactions
Workspace Resolver	Locates the repository root and current working context	Configuration, path policy, command runner
Path Guard	Normalizes paths and rejects escapes outside the workspace	File service, patch service, diff service
Filesystem Service	Reads, writes, appends, renames, and deletes files	Artifact store, audit log, tool router
Patch / Diff Service	Computes and applies file deltas	Filesystem service, coding agent, review agent
Command Runner	Executes local commands with allowed working directories	Tool router, validator, test harness
Artifact Registry	Stores command outputs, diffs, logs, and generated files	All operations, future benchmark harness
Audit Logger	Records every operation with timestamps and request IDs	All services, observability layer
Tool Adapter	Translates agent requests into internal service calls	LangGraph, future MCP wrapper



Required abstractions are deliberately small: request envelope, operation result, artifact descriptor, workspace context, and execution policy. Anything larger becomes a hidden architecture layer and belongs in a later part.

Design constraint
The services must be composable without becoming coupled. Every component should be replaceable in tests, because a local tool layer that cannot be mocked is just a future outage with better branding.

5. Project Structure

A clean directory layout matters because later agents and utilities will grow around these entry points. The structure should isolate policy, execution, schemas, state, and tests so that behavior stays understandable as the system grows.

<pre>runtime/ workspace/ # repo root discovery, cwd handling, workspace metadata filesystem/ # read/write/patch/diff operations commands/ # local command execution, allowlists, capture artifacts/ # output files, diffs, logs, manifests routing/ # tool adapter, request normalization, dispatch state/ # session context, operation history, last results schemas/ # request/response and config schemas config/ # workspace, policy, command, and artifact settings logging/ # structured events, audit formatting, log sinks sandbox/ # temp dirs, fixtures, isolated execution roots tests/ # unit, integration, safety, and regression coverage</pre>	
Folder	Why it exists
runtime/workspace	Keeps repository root handling and directory rules separate from file content operations
runtime/filesystem	Owns file reads, writes, patches, and diffs as a coherent service
runtime/commands	Contains shell execution policy and command result capture
runtime/artifacts	Stores outputs that must survive beyond a single request

runtime/routing	Adapts external requests to internal service calls
state	Tracks request-scoped and session-scoped runtime state
schemas	Defines stable request and response shapes for every tool
config	Holds policy knobs that must not be hard-coded
logging	Centralizes structured logs and audit formatting
sandbox	Provides isolated fixtures and test workspaces
tests	Verifies safety, correctness, and regression behavior

6. Data Contracts

The data contracts are the narrow waist of the implementation. Everything crosses this boundary in a structured envelope so later stages can consume the same result format regardless of whether the operation came from a human, a planner, or a future agent.

Contract	Fields	Example meaning
Request envelope	request_id, actor, operation, target, policy, timestamp	A single local action request
Workspace context	repo_root, active_path, allowed_roots, branch/worktree, session_id	Where the operation is allowed to occur
File result	status, path, bytes_read, digest, preview, error	Outcome of a read or write
Command result	status, command, cwd, exit_code, stdout_ref, stderr_ref, duration	Outcome of a local command
Artifact descriptor	artifact_id, kind, path, size, checksum, provenance	A persistent output reference
Audit event	event_id, request_id, actor, action, outcome, metadata	Trace of what happened
Config object	workspace policy, command allowlist, artifact rules, logging rules	The runtime policy surface

Pseudo-structure examples are enough here. The engineer should know the shape of the data, not receive a production implementation.

```
RequestEnvelope
{ request_id, actor, operation, target, policy, timestamp }

CommandResult
{ status, command, cwd, exit_code, stdout_ref, stderr_ref, duration }

ArtifactDescriptor
{ artifact_id, kind, path, checksum, provenance }
```

Contract rule

Every response must be machine-readable, traceable, and serializable. If a result cannot be logged and reconsumed later, it is not a valid output for this part.

7. State Management

State in this part should remain intentionally limited. The runtime owns ephemeral request state, the workspace layer owns repository context, and the artifact registry owns persisted outputs. There should be no hidden mutable global state pretending to be convenient.

State type	Owner	Persistence	Consumed by
Request state	Runtime request handler	Ephemeral	Current operation only
Workspace context	Workspace resolver	Ephemeral with reloadable config	Filesystem and command services
Session history	State store	Session-scoped	Future tool calls in the same run
Artifact manifest	Artifact registry	Persistent on disk	Review, testing, recovery, benchmark stages
Audit log	Logging layer	Persistent append-only	Debugging, traceability, verification

State changes should happen only at explicit checkpoints: after validation, after execution, after artifact capture, and after audit write. This makes recovery and replay possible. Future agents will consume these records to avoid re-reading the world from scratch.

```
State update flow
1. Receive request
2. Validate request and workspace
3. Execute operation
4. Capture result and artifacts
5. Append audit event
6. Publish normalized state snapshot
```

8. Component Specifications

Each component below should be implemented as a separately testable unit. The point is not stylistic purity. The point is to stop one broken subsystem from contaminating every other subsystem like a tired intern with root access.

Component	Purpose	Inputs	Outputs	Failure conditions	Future integration
Workspace Resolver	Establish repository context	Config, current process path, repo markers	Repo root, active workspace info	No repository found, ambiguous root, invalid config	Planner, research, worktree isolation
Path Guard	Enforce boundary policy	Requested path, repo root, allow/deny rules	Canonical safe path or rejection	Traversal attempt, symlink escape, forbidden mount	Filesystem, patch, diff, artifact services
Filesystem Service	Perform file operations	Safe path, payload, operation type	Read text, write status, diff refs	Missing file, permission denial, partial write	Coding, review, context compression
Command Runner	Execute local commands	Command spec, cwd, env policy	Exit code, stdout/stderr refs, duration	Timeout, disallowed command, nonzero exit	Testing, recovery, deployment prep
Artifact Registry	Persist outputs	Captured content, metadata, provenance	Artifact IDs and manifests	Disk full, checksum failure, write error	Benchmarking, reporting, replay
Audit Logger	Record every operation	Request envelope, action outcome, metadata	Append-only event record	Log write failure, schema mismatch	Evaluation, troubleshooting, observability
Tool Adapter	Expose stable internal tools	External request shape	Internal service call + response	Schema mismatch, unsupported operation	LangGraph agents, MCP wrapper later

The implementation should keep these components boring on purpose. A boring local tool layer is a triumph. Excitement is for feature demos and incident reports.

Implementation rule

Do not let one component call another through implicit side effects. Every interaction should be visible in code, visible in tests, and visible in logs.

9. Implementation Sequence

- Step 1: Define policy and schema boundaries - Lock down the request envelope, response envelope, config format, and workspace policy so later code has a stable contract. Dependency: Everything else depends on the shape of the inputs and outputs.
- Step 2: Implement workspace resolution - Locate the repo root, normalize paths, and create the workspace context object. Dependency: The rest of the system needs a known boundary before any file access can happen.
- Step 3: Build safe file reads - Add read-only file access with clear errors, previews, and digest support. Dependency: Research and planning stages need deterministic reads before writes are allowed.
- Step 4: Build safe writes and patches - Add atomic writes, append operations, and patch application with preflight checks. Dependency: The coding agent cannot exist without controlled modification.
- Step 5: Build diff generation - Expose file diffs as first-class outputs so review and evaluator stages can inspect change sets. Dependency: Change visibility is required before quality gates.
- Step 6: Add command runner - Implement allowlisted local command execution with working-directory control and output capture. Dependency: Testing and verification depend on trustworthy command results.
- Step 7: Add artifact registry and audit logging - Persist outputs, references, and event traces so later stages can reason over history. Dependency: Recovery, evaluation, and benchmarking require durable records.
- Step 8: Add tool adapter and integration tests - Expose the internal services through a consistent tool interface and prove the end-to-end flow. Dependency: Agents will consume this layer only after the interfaces are stable.

The build order matters because it prevents the most common failure mode: writing command execution before boundary control and then discovering too late that the tool layer can wander out of the repository like a confused raccoon.

Sequencing rule

Safety boundaries come before mutating operations. Mutating operations come before command orchestration. Artifact capture comes before higher-level agent behavior.

10. Validation Strategy

Validation should combine unit tests, integration tests, and negative-path safety checks. Every service needs tests that prove both happy-path correctness and refusal behavior under invalid input.

Subsystem	How to test	What counts as correct
Workspace resolver	Use temporary repositories and malformed root inputs	Correct root discovery and consistent rejection of invalid paths
Path guard	Try traversal, symlink escapes, and forbidden targets	No operation escapes the allowed workspace
Filesystem service	Read, write, patch, and diff sample files	Exact file contents and deterministic diffs
Command runner	Execute allowlisted commands and blocked commands	Allowed commands run; blocked commands are rejected cleanly
Artifact registry	Create artifacts and validate checksums	Artifacts are persisted with stable metadata
Audit logger	Force repeated requests and inspect records	Every operation produces a traceable log event
Tool adapter	Run end-to-end request/response fixtures	Stable schema and normalized responses

Regression prevention should rely on a small set of workspace fixtures: a tiny repository, a nested repository, a broken repository, and a sandbox with a symlink trap. Those fixtures are worth more than a hundred optimistic unit tests.

Verification loop

1. Run unit tests for path policy and file operations
2. Run integration tests against temp repositories
3. Run blocked-command tests
4. Confirm artifact outputs and audit logs
5. Re-run after any schema or policy change

11. Deliverables

Deliverable	What it should contain
Configuration system	Workspace policy, command allowlist, artifact settings, logging settings
Workspace resolver	Repository root detection and workspace context creation
Filesystem service	Safe read/write/patch/diff functionality
Command runner	Controlled local command execution and result capture
Artifact registry	Persistent storage for command outputs and file-generated outputs
Audit logger	Structured event trail for every tool action
Tool adapter	Stable interface used by agents and orchestrators
Validation suite	Unit and integration tests for safety and correctness
Sample fixtures	Small repositories and sandbox directories for testing

Deliverables should be inspectable as discrete artifacts, not just inferred from code. If an engineer cannot point to the module, the config, the tests, and the logs, the part is not done.

Artifact rule

The system is only as real as its outputs. A design without durable artifacts is a presentation, not an engineering deliverable.

12. Acceptance Criteria

Criterion	How it is measured
Workspace root is discovered consistently	Same repository resolves to the same root across repeated runs
Path traversal is rejected	Inputs outside the workspace return a structured denial
File reads are exact	Known files produce byte-for-byte correct output
Writes are atomic and visible	Written file content matches expected output after commit
Diffs are deterministic	Same input change set produces the same diff representation
Commands are controlled	Only allowlisted commands execute successfully
Command output is captured	Stdout, stderr, exit code, and duration are returned
Artifacts are persisted	Generated outputs are stored and retrievable by ID
Audit trail is complete	Every operation leaves a traceable event record
Tests are green	All unit, integration, and safety tests pass in CI

Acceptance should be strict enough that an engineer cannot claim completion based on a demo that worked once on a lucky laptop. Reality has enough fans already.

Completion threshold

Part 2 is accepted only when the implementation can be exercised repeatedly on clean temporary repositories and produces the same safe behavior every time.

13. Common Failure Modes

Failure mode	What it looks like	How to avoid it
Path traversal bug	File operations unexpectedly escape the repo root	Normalize and validate every path before use

Command injection	User input reaches the shell unsafely	Use structured command specs and allowlists
Silent partial writes	Files are left in broken intermediate states	Use atomic write patterns and post-write verification
Inconsistent working directory	Commands run in the wrong folder	Make cwd explicit in every execution request
Unlogged side effects	Operations happen without a durable trace	Log before and after execution with request IDs
Overgrown tool interface	The adapter becomes a dumping ground for unrelated behaviors	Keep the interface narrow and stable
Artifact sprawl	Outputs exist but cannot be found later	Use a manifest and stable artifact IDs
Tight coupling	Services call each other through hidden assumptions	Inject dependencies explicitly and test them separately

The design traps are predictable. People add one shortcut, then another, then they are somehow debugging a shell wrapper that also decides policy, stores memory, and writes poetry. Resist the urge.

Anti-pattern

Do not add agent planning, retrieval, memory, or evaluation logic to the filesystem layer. The second that happens, the part stops being a local tooling layer and becomes an unmaintainable blob.

14. Future Integration

Future part	How Part 2 connects	Why it matters now
Part 3 Planning Layer	Planner consumes file context and command availability	Planning needs precise tool boundaries
Part 4 Research Layer	Research reads docs, configs, and repository files safely	Research needs reliable read operations
Part 5 Memory System	Workspace and artifact metadata become retrievable state	Persistent records depend on structured outputs
Part 6 Testing & Review	Testing agent runs commands and inspects artifacts	Automated verification depends on command capture
Part 7 Evaluation Layer	Evaluator compares task intent to file and command outcomes	Evaluation needs complete execution traces
Part 8 Failure Recovery	Recovery inspects logs, artifacts, and exit codes	Retry strategy needs observability
Part 9 Repository Intelligence	Indexers read repository contents and artifacts	Intelligence depends on clean repository access
Part 10 Code Knowledge Graph	Graph builder traverses file relationships and metadata	Graph ingestion begins with trustworthy file access

Every later stage assumes Part 2 as a substrate. That means this part must stay boring, stable, and explicit. If it becomes clever, every later part inherits cleverness in the worst possible way.

Part 2 outputs used later:

- safe file reads
- controlled writes and patches
- deterministic diffs
- command exit data
- artifact IDs
- audit events

15. Completion Checklist

- ☐ Repository root discovery works on clean, nested, and invalid repository layouts.
- ☐ Path normalization blocks traversal, symlink escape, and forbidden targets.
- ☐ File read operations return exact content and structured errors.
- ☐ Write and patch operations are atomic and verified after execution.
- ☐ Diff generation produces deterministic output for the same change set.
- ☐ Command runner supports allowlisted commands with explicit working directory control.
- ☐ Stdout, stderr, exit code, duration, and artifact references are captured for every command.
- ☐ Artifact registry persists outputs with stable IDs and checksums.
- ☐ Audit logger records every request and result.
- ☐ Tool adapter exposes a stable request and response contract.
- ☐ Unit, integration, and negative-path tests all pass.
- ☐ A fresh engineer can understand the directory structure and module responsibilities without reading implementation code.

Definition of done

When the final checklist is complete, Part 2 can serve as the machine's local hands and eyes without becoming its brain. That distinction matters, annoyingly enough.